Principles of Robot Autonomy I
Homework 1
Due Thursday, October 8 (11:59pm PT)

Several different software tools will be utilized throughout the course. To complete this assignment, make sure you have the following tools installed on your computer:

1. Git: a version control system for software development, an essential tool for software collaboration.
2. Python version 3.5+
3. Jupyter Notebook: A web application for interactive code development and prototyping.

Starter code for this homework has been made available online through GitHub. To get started, download the code by running `git clone https://github.com/PrinciplesofRobotAutonomy/AA274a-HW1-F26` in a terminal window.

You will submit your homework to Gradescope. Your submission will consist of a single pdf with your answers for written questions and relevant plots from code.

Your submission must be typeset in LaTeX.

You may discuss the problems with your classmates, but you must prepare your solutions on your own and list the names of the classmates you worked with. Using generative AI tools to substantially complete the assignment is not permitted, and any use beyond incidental use must be acknowledged in your submission. See the course outline for the full policy.

Introduction

The goal of this homework is to familiarize you with algorithms for path planning in constrained environments (e.g. in the presence of obstacles) and techniques to integrate planning with trajectory generation.

Note that this homework represents the start of an incremental journey to build our robot autonomy stack. There is a mix of code that is to be implemented in Python and Jupyter Notebooks, and some ROS2 integrated code that will need to run with Gazebo.

Problem 1: A* Motion Planning & Path Smoothing [30 Points]

We will implement an A^* algorithm for motion planning, as outlined in pseudocode in Algorithm 1. In particular, we will apply this algorithm to plan discrete paths on a 2D grid (state $\mathbf{x} = (x, y)$).

Note: Execute in your Linux environment using the system python. Ensure Jupyter is installed (if not, run `sudo apt install jupyter`) and install the dependencies listed in `requirements.txt`.

(i) 🖳 (**5 Points**) Implement the remaining functions in `P1_astar.py` within the `AStar` class. These functions represent many of the key functional blocks at play in motion planning algorithms:

Algorithm 1 A^* Motion Planning

Require: $\mathbf{x}_{\text{init}}$, $\mathbf{x}_{\text{goal}}$

$\mathcal{O}$.INIT($\mathbf{x}_{\text{init}}$) ▷ Open set initialized with $\mathbf{x}_{\text{init}}$
$\mathcal{C}$.INIT($\emptyset$) ▷ Closed set is initially empty
SET_COST_TO_ARRIVE_SCORE($\mathbf{x}_{\text{init}}, 0$)
SET_EST_COST_THROUGH($\mathbf{x}_{\text{init}}$, HEURISTIC($\mathbf{x}_{\text{init}}$))
while $\mathcal{O}$.SIZE > 0 **do**
    $\mathbf{x}_{\text{current}} \leftarrow$ LOWEST_EST_COST_THROUGH($\mathcal{O}$)
    if $\mathbf{x}_{\text{current}} = \mathbf{x}_{\text{goal}}$ **then**
        return RECONSTRUCT_PATH
    end if
    $\mathcal{O}$.REMOVE($\mathbf{x}_{\text{current}}$)
    $\mathcal{C}$.ADD($\mathbf{x}_{\text{current}}$)
    for $\mathbf{x}_{\text{neigh}}$ **in** NEIGHBORS($\mathbf{x}_{\text{current}}$) **do**
        if $\mathbf{x}_{\text{neigh}}$ **in** $\mathcal{C}$ **then**
            continue
        end if
        tentative_cost_to_arrive = GET_COST_TO_ARRIVE($\mathbf{x}_{\text{current}}$)+DISTANCE($\mathbf{x}_{\text{current}}, \mathbf{x}_{\text{neigh}}$)
        if $\mathbf{x}_{\text{neigh}}$ **not in** $\mathcal{O}$ **then**
            $\mathcal{O}$.ADD($\mathbf{x}_{\text{neigh}}$)
        else if tentative_cost_to_arrive $>$ GET_COST_TO_ARRIVE($\mathbf{x}_{\text{neigh}}$) **then**
            continue
        end if
        SET_CAME_FROM($\mathbf{x}_{\text{neigh}}, \mathbf{x}_{\text{current}}$)
        SET_COST_TO_ARRIVE($\mathbf{x}_{\text{neigh}}$, tentative_cost_to_arrive)
        SET_EST_COST_THROUGH($\mathbf{x}_{\text{neigh}}$, tentative_cost_to_arrive + HEURISTIC($\mathbf{x}_{\text{neigh}}$))
    end for
end while
return Failure

- `is_free`: which checks whether a state is collision-free and valid.
- `distance`: which computes the Euclidean travel distance between two points. This is the edge cost used for the cost-to-arrive.
- `heuristic`: which computes the (weighted) estimate $w \|\mathbf{x} - \mathbf{x}_{\text{goal}}\|$ of the cost-to-go, where the norm ($\mathcal{L}^1$, $\mathcal{L}^2$, or $\mathcal{L}^\infty$) and the weight $w \geq 1$ are set when the `AStar` object is constructed.
- `get_neighbors`: which finds the free neighbor states of a given state.
- `solve`: which runs the A^* motion planning algorithm.

Be sure to read the documentation for every function for a more detailed description. You can test this implementation in a couple of planning environments. To do so, open the associated Jupyter notebook by running the following command:

```
$ jupyter notebook sim_astar.ipynb
```

Make sure all assertions in the "Sanity checks" section pass. Please include the plots from the "Simple Environment" and "Random Cluttered Environment" sections of the notebook in your write-up. In the "Random Cluttered Environment" section, feel free to play with the number of obstacles and other parameters of the randomly generated environment, but generate the plots for your write-up with the default parameters and `SEED = 274`.

Note: Notice that we collision-check states but do not collision-check edges. This saves us some computation (collision-checking is often one of the most expensive operations in motion planning). Also, in this case the obstacles are aligned with the grid, so paths will remain collision-free. However,

outside such special circumstances one should add edge collision-checking and/or inflate obstacles to guarantee collision-avoidance.

(ii) ⌨ (**5 points**) In the final segment of Problem 1, we transition from the geometric paths obtained from the A* algorithm to generating feasible trajectories for our differential drive robot.

Smooth the paths from A* by fitting a cubic spline to the path nodes. Implement this within the `compute_smooth_plan` function of `sim_astar.ipynb`. You may need to use the `splrep` function from `scipy.interpolate` (read through the documentation to understand its usage and parameters).

Since all we have is a geometric path, you should estimate the time for each of the points assuming that we travel at a fixed speed v_{des} along each segment. Compute the cumulative time along the path waypoints and use it for spline fitting.

Adjust the smoothing parameter α (denoted s in `splrep`) to strike a balance between following the original collision-free trajectory and risking collision for additional smoothness. Keep $v_{des} = 0.15$ m/s.

Please include the plot generated in the "Smooth Trajectory" section of the notebook in your write-up, and state the value of α you chose. In 2–3 sentences, explain what happens to the smoothed path as α becomes very large, and why this is dangerous in the random cluttered environment.

Note: There are many ways to ensure smoothed solutions are collision-free (e.g. collision-checking smoothed paths and running a dichotomic search on α to find a tight fit against obstacles, or inflating obstacles in the original planning to give additional room for smoothing). This strategy can be used on geometric sampling-based planning methods as well.

(iii) ✎ (**20 Points**) Now we will study how the choice of heuristic affects our planning results. Note that throughout this part the edge cost (`distance`) remains Euclidean; only the `heuristic` changes. Run the "Heuristic Comparison" section of the notebook, which plans on the random cluttered environment (`SEED = 274`) using the $\mathcal{L}^2$, $\mathcal{L}^1$, and $\mathcal{L}^\infty$ heuristics with $w = 1$, and the $\mathcal{L}^2$ heuristic with $w = 3$. Include the resulting plot and table in your write-up, and answer the following.

(a) (**6 Points**) Recall that a heuristic h is *admissible* if $h(\mathbf{x}) \leq c^*(\mathbf{x})$ for every state $\mathbf{x}$, where $c^*(\mathbf{x})$ is the true optimal cost-to-go from $\mathbf{x}$ to $\mathbf{x}_{\text{goal}}$. For our 8-connected grid with Euclidean edge costs, determine which of the three heuristics ($w = 1$) are admissible. Justify each answer; for any heuristic that is not admissible, give a concrete pair of states $\mathbf{x}$, $\mathbf{x}_{\text{goal}}$ for which it overestimates the cost-to-go.

(b) (**6 Points**) Using your answer to (a), explain the ordering of the number of nodes expanded by the three $w = 1$ heuristics in your table. Why does a heuristic that is larger (but still admissible) generally lead to fewer expansions?

(c) (**8 Points**) Weighted A^* with a consistent (and hence admissible) heuristic h, such as the $\mathcal{L}^2$ heuristic here, and weight $w \geq 1$ is guaranteed to return a path whose cost is at most w times the optimal cost. Compare the path length and the number of nodes expanded for the $\mathcal{L}^2$ heuristic with $w = 1$ and $w = 3$, and verify that the bound holds for your result. Finally, since the $\mathcal{L}^1$ heuristic satisfies $\|\mathbf{x}\|_1 \leq \sqrt{2}\,\|\mathbf{x}\|_2$ in 2D, what bound on its path cost relative to the optimal cost can you conclude? Is the $\mathcal{L}^1$ result in your table consistent with this bound?

Problem 2: Rapidly-exploring Random Trees (RRT) [20 Points]

While our A^* planning relies on a predefined set of viable samples on the edges of a graph, in some scenarios it is useful to draw samples incrementally and in a less structured fashion. This motivates sampling-based algorithms such as Rapidly-exploring Random Trees (RRT), which we will implement in this problem.

Since vanilla RRT builds its tree by extending from the nodes nearest to random samples, we cannot add the same heuristic as A^* to bias search in the direction of the goal. Instead, we will use a goal-biasing approach, included in the pseudocode in Algorithm 2.

Algorithm 2 RRT with goal biasing.

Require: $\mathbf{x}_{\text{init}}$, $\mathbf{x}_{\text{goal}}$, maximum steering distance $\varepsilon > 0$, iteration limit K, goal bias probability $p \in [0, 1]$

$\mathcal{T}$.INIT($\mathbf{x}_{\text{init}}$)
for $k = 1$ to K **do**
  Sample $z \sim$ Uniform($[0, 1]$)
  if $z < p$ **then**
    $\mathbf{x}_{\text{rand}} \leftarrow \mathbf{x}_{\text{goal}}$
  else
    $\mathbf{x}_{\text{rand}} \leftarrow$ RANDOM_STATE()
  end if
  $\mathbf{x}_{\text{near}} \leftarrow$ NEAREST_NEIGHBOR($\mathbf{x}_{\text{rand}}, \mathcal{T}$)
  $\mathbf{x}_{\text{new}} \leftarrow$ STEER_TOWARDS($\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{rand}}, \varepsilon$)
  if COLLISION_FREE($\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{new}}$) **then**
    $\mathcal{T}$.ADD_VERTEX($\mathbf{x}_{\text{new}}$)
    $\mathcal{T}$.ADD_EDGE($\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{new}}$)
    if $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{goal}}$ **then return** $\mathcal{T}$.PATH($\mathbf{x}_{\text{init}}, \mathbf{x}_{\text{goal}}$)
    end if
  end if
end for
return Failure

(i) ⌨ (**8 Points**) Implement RRT for 2D geometric planning problems (state $\mathbf{x} = (x, y)$) by filling in `RRT.solve`, `GeometricRRT.find_nearest`, and `GeometricRRT.steer_towards` in `P2_rrt.py`. In addition to the tree and the solution path, `RRT.solve` must record in `self.num_iters` the number of iterations it ran before returning (see the comments in the code).

You can validate your implementations of the parts of this problem in the associated notebook:

```
$ jupyter notebook sim_rrt.ipynb
```

Please include the generated plot from the "Geometric Planning" section of the notebook in your write-up, generated with the default start, goal, and `SEED = 274`.

(ii) You may have noticed that due to the random sampling in RRT, there is plenty of room to optimize the length of the resulting paths. This motivates a variety of post-processing methods which locally optimize motion planning paths. As it turns out, even very simple methods can perform quite well on this task. We will implement one of the simplest of these algorithms, which we simply call Shortcut

⌨ (**7 Points**) Implement the shortcutting algorithm outlined in the pseudocode in Algorithm 3 by filling in `RRT.shortcut_path`. You can test your implementation in the notebook and should notice that in nearly all cases, Shortcut will be able to refine to a shorter path.

Please include the generated plot from the "Adding shortcutting" section of the notebook in your write-up.

Note: Post-processing algorithms such as this are performing a *local* optimization, which means the result may be far from a globally optimal path. For example in this case, shortcutting is not likely to move the path to the other side of an obstacle (i.e. to a different solution homotopy class), even if this would result in lower path length. This motivates the use of asymptotically optimal varieties of sampling-based planners such as RRT*, which perform a *global search* and are thus guaranteed to approach the globally optimal solution.

(iii) ✎ (**5 Points**) Run the "Goal-bias experiment" section of the notebook, which runs your RRT 20 times for each goal-bias probability $p \in \{0, 0.05, 0.2, 0.5, 0.9\}$. Include the resulting plot and table in your write-up, and answer the following:

Algorithm 3 Shortcut (deterministic)

Require: $\Pi_{\text{path}} = (\mathbf{x}_{\text{init}}, \mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_{\text{goal}})$
SUCCESS = False
while not SUCCESS **do**
  SUCCESS = True
  for $\mathbf{x}$ **in** Π_{path} where $\mathbf{x} \neq \mathbf{x}_{\text{init}}$ and $\mathbf{x} \neq \mathbf{x}_{\text{goal}}$ **do**
    if COLLISION_FREE(PARENT($\mathbf{x}$), CHILD($\mathbf{x}$)) **then**
      Π_{path}.REMOVE_NODE($\mathbf{x}$)
      SUCCESS = False
    end if
  end for
end while

(a) Explain why RRT (as implemented in Algorithm 2) essentially never succeeds when $p = 0$, even though the tree keeps growing throughout the free space.

(b) Explain why the success rate drops again for large p in this maze. Refer to the specific obstacles between $\mathbf{x}_{\text{init}}$ and $\mathbf{x}_{\text{goal}}$ in your answer.

Problem 3: Heading Controller (Section Prep) [20 Points]

Objective: Develop a ROS2 node for heading control using a proportional controller. This controller aims to minimize the error between the current heading of the TurtleBot3 robot and a desired goal heading. You'll be using ROS2 (Robot Operating System) with the rclpy library, utilizing the given messages and utility functions.

Background:

- `rclpy`: Python library to write ROS2 nodes.
- `TurtleBotState`: A message type that contains state information of the TurtleBot3, including its heading (theta).
- `TurtleBotControl`: A message type that contains control commands for the TurtleBot3, including angular velocity (omega).
- `wrap_angle`: A utility function that wraps angles between $-\pi$ and π.

Instructions:

1. Workspace Setup:
   - Open your Ubuntu installation on your Linux environment. From the home directory, create directory `~/autonomy_ws/src`:

```
$ mkdir -p ~/autonomy_ws/src
```

   - From the `HW1` folder in the homework repository, move the `autonomy_repo` folder in to `autonomy_ws/src`.
   - Navigate to the `~/autonomy_ws` directory, build the workspace, and source:

```
$ cd ~/autonomy_ws
$ colcon build --symlink-install
$ source install/local_setup.bash
```

- Navigate to `~/autonomy_ws/src/autonomy_repo/scripts/heading_controller.py`. This is the file you will be editing in the next steps, and should be completely blank when you start.

2. Initial Imports:

   - Set the shebang at the top of your Python file: `#!/usr/bin/env python3`.
   - Import necessary libraries: numpy and rclpy.
   - From the `asl_tb3_lib.control` package, import the `BaseHeadingController` module.
   - From the `asl_tb3_lib.math_utils` package, import `wrap_angle`.
   - From the `asl_tb3_msgs.msg` package, import `TurtleBotControl` and `TurtleBotState` messages.

3. Define the `HeadingController` Class:

   - Create a class `HeadingController` that inherits from `BaseHeadingController`.
   - In the `__init__` method
     - Call the parent's `__init__` method.
     - Define a class variable for the proportional control gain `kp` and set it to 2.0.

4. Proportional Control:

   - Inside your `HeadingController` class, you should override the `compute_control_with_goal()` method from the `BaseHeadingController` class. This method is left unimplemented in the base class and serves as a placeholder for you to define your heading control logic.
     - The method takes in the current state and the desired state of the TurtleBot, both of which are of type `TurtleBotState`.
     - It should return a control message of type TurtleBotControl.
     - Use type annotations in your method signature to enforce these types.
   - Inside this method calculate the heading error ($\in [-\pi, \pi]$) as the wrapped difference between the goal's theta and the state's theta.
   - Use the proportional control formula, $\omega = k_p \cdot \text{err}$, to compute the angular velocity required for the TurtleBot to correct its heading error.
   - Create a new `TurtleBotControl` message, set its `omega` attribute to the computed angular velocity, and return it.

5. Node Execution:

   - In the main block (`if __name__ == "__main__":`) initialize the ROS2 system using `rclpy.init()`.
   - Create an instance of the `HeadingController` class.
   - Spin the node using `rclpy.spin()` to keep it running and listening for messages.
   - Ensure to shut down the ROS2 system with `rclpy.shutdown()` after spinning.

6. Running with your Simulator:

```
# Open three terminals, and run the following in each of them.
$ cd ~/autonomy_ws
$ source install/local_setup.bash

# In Terminal 1 run
$ ros2 launch asl_tb3_sim root.launch.py
# This will start your Turtlebot simulator
# no GUI should appear.

# In Terminal 2 run
$ ros2 launch autonomy_repo heading_control.launch.py
# This will start up the heading controller that you
# just created, and open RVIZ. DO NOT SET A GOAL POSE.
# Allow thirty seconds for the code in Terminal 2 to fully start
   up.

# In Terminal 3 run.
$ ros2 run autonomy_repo p3_plot.py
# This will command a fixed goal position to test your controller,
# and produce a plot that you can submit for grading.
# You should see your Turtlebot moving in RVIZ.
# This function will take at least ten seconds to produce a plot.

# After you've successfully produced a plot,
# try setting a new goal post in RVIZ by selecting the
# "Goal Pose" button on the top toolbar and then
# clicking and dragging in the environment to set
# a goal heading.
```

7. (**10 Points**) Please include the resulting plot `p3_output.png` in your write-up.

8. (**10 Points**) Let's analyze how the gain k_p shapes the closed-loop behavior. Model the heading dynamics as $\dot{\theta} = \omega$ with the control law $\omega = k_p e$, where $e = \theta_{\text{goal}} - \theta$ and θ_{goal} is constant (ignore angle wrapping).

   (a) (**3 Points**) Ignoring any limits on ω, show that the error evolves as $e(t) = e(0)\, e^{-k_p t}$. The test script commands $\theta_{\text{goal}} = 1.274$ rad. Using the initial heading in your plot and $k_p = 2$, compute the time it takes for the error to drop to 5% of its initial value.

   (b) (**3 Points**) `BaseHeadingController` runs its control loop at 10 Hz and holds ω constant between updates, so with $\Delta t = 0.1$ s the error evolves as $e_{k+1} = (1 - k_p \Delta t)\, e_k$. For what range of k_p does the error converge to zero without changing sign? For what range does it converge at all? Describe what you would expect to see the robot do when k_p is above each of these thresholds.

   (c) (**4 Points**) `BaseHeadingController` also clips the commanded angular velocity to $|\omega| \leq \omega_{\max} = 0.4$ rad/s (the `om_max` parameter). With $k_p = 2$, for which heading errors is ω saturated? Describe the two phases of the resulting response, identify them in your plot, and state the heading θ at which your plot should switch from one to the other. Use this to explain why your plot takes much longer to converge than your prediction in part (a). Finally, give two reasons why k_p cannot be made arbitrarily large on a physical robot.